KronoDroid Final Year Project

Complete 3-Member Core Role Assignment

Project Role Structure

The project is divided into three substantial technical cores. Each member owns a complete research component with implementation, experiments, analysis, and deliverables. Dataset preparation is treated as shared infrastructure rather than one member's primary role.

Member	Core Technical Role
Member 1	Malware Feature Representation & Baseline Modeling
Member 2	Few-Shot Embedding & Metric Learning
Member 3	Open-Set Malware Family Recognition & Few-Shot Adaptation

1. Member 1 — Malware Feature Representation & Baseline Modeling

Core responsibility: Find and establish the best malware feature representation for the few-shot system. This role is not limited to preprocessing.

Core research question: Which KronoDroid feature representation is most effective for fine-grained malware-family recognition?

Tasks

- Work with static, dynamic/system-call, hybrid, family-label and relevant temporal information.
- Create and compare Static, Dynamic and Hybrid feature configurations.
- Perform feature selection and feature-ablation studies using appropriate methods.
- Study which feature groups contribute most to malware-family discrimination.
- Perform relevant temporal/device-wise analysis where applicable.
- Build conventional machine-learning baselines such as Logistic Regression, Random Forest and SVM.
- Compare baseline performance across Static, Dynamic and Hybrid representations.
- Select and document the final feature representation passed to the few-shot model.

Experiments: Static vs Dynamic vs Hybrid; feature-selection/ablation; baseline model comparison; relevant temporal/device comparisons.

Deliverables: feature pipeline, feature-selection code, baseline models, baseline results, feature-analysis notebooks, and the final feature configuration.

2. Member 2 — Few-Shot Embedding & Metric Learning

Core responsibility: Develop the model that converts malware features into a discriminative embedding space.

Core research question: Can a discriminative malware-family embedding be learned effectively when only a small number of labelled samples are available?

Tasks

- Implement the proposed PyTorch embedding network.
- Use the planned dense architecture with a 64-dimensional normalized embedding.
- Implement episodic N-way K-shot training.

- Implement the prototypical/metric-learning objective.
- Train the embedding so same-family samples become closer and different-family samples become more separable.
- Run 1-shot, 5-shot, 10-shot and 20-shot few-shot experiments.
- Perform model and hyperparameter experiments such as embedding dimension, hidden-layer configuration, dropout and learning rate.
- Produce and version the trained embedding-model checkpoint for Member 3.

Experiments: 1-shot vs 5-shot vs 10-shot vs 20-shot; embedding/model configuration experiments; training and validation analysis.

Deliverables: embedding network, episode sampler, prototypical/metric-learning loss, training pipeline, configuration files, trained model checkpoint and training results.

3. Member 3 — Open-Set Malware Family Recognition & Few-Shot Adaptation

Core responsibility: Turn the learned embedding into a system that can recognize known families, reject unseen families, and learn those unseen families from a few labelled examples.

Core research question: Can the system detect a completely unseen malware family and adapt to recognize it using only a few labelled samples?

Tasks

- Construct family prototypes from support-set embeddings.
- Implement query-to-prototype distance calculation and known-family classification.
- Develop the distance-based Known vs UNKNOWN decision mechanism.
- Determine and validate the unknown-family threshold instead of using an arbitrary fixed value.
- Design and run strict unseen-family experiments in which complete families are withheld from base training.
- Implement few-shot adaptation for a newly discovered family.
- Create a new prototype from 1, 5, 10 or 20 labelled support samples and evaluate subsequent recognition.
- Evaluate known-family classification, unknown-family detection and post-adaptation performance.

Experiments: threshold selection; strict unseen-family detection; 1/5/10/20-shot adaptation; known-family and open-set metrics.

Deliverables: prototype module, distance module, unknown detector, threshold-selection method, adaptation module, inference pipeline and final evaluation results.

Integration Between the Three Members

From → To	Interface
Member 1 → Member 2	Final feature representation + train/support/query data. Member 2 does not depend on raw-data cleaning
Member 2 → Member 3	Versioned embedding_model.pt + stable embedding function/interface.
Member 3 → Final System	Known-family prediction + UNKNOWN detection + few-shot adaptation.

Overall Pipeline

KronoDroid → Feature Representation & Baselines → Few-Shot Embedding → Family Prototypes → Distance-Based Decision → KNOWN / UNKNOWN → Few-Shot Adaptation → Final Evaluation

Workload Principle

All three members must have an independent technical contribution. Each member is expected to produce code, run controlled experiments, analyze results, document findings, and understand the interfaces to the other modules. No member should be assigned only dataset cleaning, documentation, presentation design, or UI work.

Final Role Summary

Member 1	What feature representation works best?
Member 2	How can we learn useful malware embeddings with few labels?
Member 3	Can we detect and adapt to completely unseen malware families?